

ZENOVA

Movie Search App

Complete Project Documentation

React + Tailwind + React Router + Custom Hooks + OMDB API

Developer: Umair Khan | React + Vite + Tailwind | MERN Stack Journey | ZENOVA

1. PROJECT OVERVIEW

Project Name	ZENOVA Movie Search App
Technology	React + Vite + Tailwind CSS + React Router + Custom Hooks
API Used	OMDB API (Open Movie Database) — Free plan 1000 req/day
Pages	Home (Search), Favorites, About ZENOVA
Developer	Umair Khan — MERN Stack Journey
Purpose	Stage 3 React project — learn real world React patterns

2. FILE STRUCTURE

```
movie-app/  
  ■■■■ src/  
    ■ ■■■■ components/  
    ■ ■ ■■■■ Navbar.jsx ← Navigation bar (shows on ALL pages)  
    ■ ■■■■ hooks/  
    ■ ■ ■■■■ useFetchMovies.js ← Custom Hook (all fetch logic lives here)  
    ■ ■■■■ pages/  
    ■ ■ ■■■■ Home.jsx ← Main page – search movies (UI only)  
    ■ ■ ■■■■ Favorites.jsx ← Favorites page  
    ■ ■ ■■■■ About.jsx ← About ZENOVA page  
    ■ ■■■■ App.jsx ← Root component – sets up routing  
    ■ ■■■■ main.jsx ← Entry point – renders App into HTML  
    ■ ■■■■ index.css ← Tailwind CSS import  
    ■■■■ .env ← API key stored here (NEVER push to GitHub!)  
    ■■■■ .gitignore ← Ignores .env and node_modules  
    ■■■■ vite.config.js ← Vite + Tailwind configuration
```

3. COMPLETE CODE FLOW DIAGRAM

```
STEP 1: Browser opens localhost:5173
|
v
STEP 2: main.jsx – Entry Point
| Renders <App /> into index.html div#root
|
v
STEP 3: App.jsx – Router Setup
| <BrowserRouter> wraps entire app
| <Navbar /> renders on ALL pages
| <Routes> decides which page to show:
| URL = / --> <Home />
| URL = /favorites --> <Favorites />
| URL = /about --> <About />
|
v
STEP 4: Navbar.jsx – Navigation
| useLocation() checks current URL
| Highlights active link (white = active, gray = inactive)
| <Link> tags change URL without page reload
|
v
STEP 5: Home.jsx – Main Page (UI ONLY)
| Calls useFetchMovies('Marvel') custom hook
| Gets back: { movies, loading, error, fetchMovies }
| Renders search input + movie grid
| useRef controls input focus
|
v
STEP 6: useFetchMovies.js – Custom Hook (LOGIC ONLY)
| useState: movies=[], loading=false, error=''
| useEffect: runs fetchMovies('Marvel') on page load
| fetchMovies(query): sends request to OMDB API
| Returns { movies, loading, error, fetchMovies }
|
v
STEP 7: OMDB API – External Service
| URL: https://www.omdbapi.com/?s=Marvel&apikey=KEY
| Returns JSON: { Response: 'True', Search: [...movies] }
|
v
STEP 8: State Updates
| setMovies(data.Search) --> React re-renders Home.jsx
| Movie cards appear on screen
```

4. USER INTERACTION FLOW

```
User types 'Batman' in search box
|
v
onChange fires --> setSearch('Batman') --> search state updates
|
v
User clicks Search button OR presses Enter key
|
v
searchMovies() runs
|
|--> if (!search) return [STOPS if input is empty]
|
v
fetchMovies('Batman') called from custom hook
|
v
setLoading(true) --> 'Loading...' appears on screen
setError('') --> clears any previous error
|
v
await fetch('https://www.omdbapi.com/?s=Batman&apikey=KEY')
|
v
await response.json() --> opens envelope, converts to JS object
|
|-- data.Response === 'True'
| |
| v
| setMovies(data.Search) --> movie cards appear!
|
|-- data.Response === 'False'
| |
| v
| setError('No movies found!')
|
|-- Network error (catch block)
| |
| v
| setError('Something went wrong!')
|
v
setLoading(false) --> 'Loading...' disappears
searchRef.current.focus() --> input stays focused
```

5. FILE BY FILE CODE EXPLANATION

main.jsx — Entry Point

```
// Line 1: Import React's rendering engine
import ReactDOM from 'react-dom/client'

// Line 2: Import root component
import App from './App'

// Line 3: Import Tailwind CSS
import './index.css'

// Line 4: Find div#root in index.html and render App inside it
ReactDOM.createRoot(document.getElementById('root')).render(<App />)
```

App.jsx — Router Setup

```
// Line 1: Import routing tools from react-router-dom
import { BrowserRouter, Routes, Route } from 'react-router-dom'
// BrowserRouter --> enables routing for entire app
// Routes --> container that holds all Route definitions
// Route --> maps URL path to a component

// Line 2-4: Import Navbar and all 3 pages
import Navbar from './components/Navbar'
import Home from './pages/Home'
import Favorites from './pages/Favorites'
import About from './pages/About'

// Line 5: App component – wraps everything in BrowserRouter
function App() {
  return (
    <BrowserRouter> // enables routing
    <div className='min-h-screen bg-gray-900 text-white'>
    <Navbar /> // shows on ALL pages
    <Routes> // only ONE route renders
    <Route path='/' element={<Home />} />
    <Route path='/favorites' element={<Favorites />} />
    <Route path='/about' element={<About />} />
    </Routes>
    </div>
    </BrowserRouter>
  )
}
```

Navbar.jsx — Navigation Bar

```

// Line 1: Import Link and useLocation from react-router-dom
import { Link, useLocation } from 'react-router-dom'
// Link --> like <a> tag but NO page reload
// useLocation --> tells us current URL path

function Navbar() {
  // Line 2: Get current URL path
  const location = useLocation()
  // Example: location.pathname = '/favorites'

  return (
    <nav>
      // ZENOVA logo links to home page
      <Link to='/'>ZENOVA</Link>

      // Each link checks if it's active
      // Active = white bold text
      // Inactive = gray text
      <Link to='/'
        className={location.pathname === '/' ? 'text-white font-bold' : 'text-gray-400'}
      >Home</Link>

      <Link to='/favorites'
        className={location.pathname === '/favorites' ? 'text-white font-bold' : 'text-gray-400'}
      >Favorites</Link>
    </nav>
  )
}

```

useFetchMovies.js — Custom Hook

```

// Custom Hook = reusable logic extracted from component
// Rule: name must start with 'use'
// Rule: can use other hooks inside (useState, useEffect)

import { useState, useEffect } from 'react'

const API_KEY = import.meta.env.VITE_API_KEY // from .env file

function useFetchMovies(defaultQuery = 'Marvel') {

  // Step 1: States – movies, loading, error
  const [movies, setMovies] = useState([])
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState('')

  // Step 2: Fetch function
  const fetchMovies = async (query) => {
    setLoading(true) // show loading
    setError('') // clear error
    try {
      const response = await fetch(`...omdbapi.com/?s=${query}&apikey=${API_KEY}`)
      const data = await response.json() // open envelope
      if (data.Response === 'True') {
        setMovies(data.Search) // save movies
      } else {
        setError('No movies found!')
      }
    } catch {
      setError('Something went wrong!')
    }
    setLoading(false) // hide loading
  }

  // Step 3: Run on page load
  useEffect(() => {
    fetchMovies(defaultQuery) // load default movies
  }, [])

  // Step 4: Return everything Home.jsx needs
  return { movies, loading, error, fetchMovies }
}

export default useFetchMovies

```

Home.jsx — Main Page (UI Only)

```
// Home.jsx only handles UI — no fetch logic here!
import { useState, useRef } from 'react'
import useFetchMovies from '../hooks/useFetchMovies'

function Home() {

  // Step 1: Get everything from custom hook in ONE LINE
  const { movies, loading, error, fetchMovies } = useFetchMovies('Marvel')
  // movies --> array of movie objects from API
  // loading --> true/false to show Loading text
  // error --> error message string
  // fetchMovies --> function to call when user searches

  // Step 2: Local states (belong to UI not fetch logic)
  const [search, setSearch] = useState('') // input value
  const searchRef = useRef(null) // input focus control

  // Step 3: Search function
  const searchMovies = () => {
    if (!search) return // stop if empty
    fetchMovies(search) // call hook's function
    searchRef.current.focus() // keep focus on input
  }

  // Step 4: UI
  return (
    <div>
      <input ref={searchRef} onChange={e => setSearch(e.target.value)} />
      <button onClick={searchMovies}>Search</button>
      {loading && <p>Loading...</p>}
      {error && <p>{error}</p>}
      {movies.map(movie => (
        <div key={movie.imdbID}>
          <img src={movie.Poster} />
          <h3>{movie.Title}</h3>
          <p>{movie.Year}</p>
        </div>
      ))}
    </div>
  )
}
```

6. STATE FLOW — HOW DATA MOVES

<code>movies = []</code>	Page loads — empty grid, no movies yet
<code>loading = true</code>	Fetch started — 'Loading...' text appears
<code>loading = false</code>	Fetch done — 'Loading...' disappears
<code>movies = [...]</code>	API returned data — movie cards appear on screen
<code>error = 'msg'</code>	Something went wrong — error message shown in red
<code>error = ''</code>	No error — error message hidden
<code>search = ''</code>	Input is empty — Search button does nothing
<code>search = 'Batman'</code>	User typed — Search button fetches Batman movies

7. KEY REACT CONCEPTS USED IN THIS PROJECT

useState	Stores movies, loading, error, search — re-renders UI on change
useEffect	Runs fetchMovies('Marvel') ONCE when page first loads
useRef	Remote control for search input — auto focus and keep focus
Custom Hook	useFetchMovies.js — separates fetch logic from UI
React Router	BrowserRouter, Routes, Route — multiple pages without reload
Link	Replaces <a> tag — changes URL without page reload
useLocation	Tells Navbar which page is active — highlights correct link
Async/Await	Waits for OMDB API response before updating state
Ternary	{loading && Loading } — shows/hides elements
Array.map()	Loops through movies array — creates one card per movie
Tailwind CSS	Utility classes for styling — no separate CSS file needed
ENV Variables	VITE_API_KEY in .env — keeps API key out of GitHub

8. IMPORTANT RULES TO REMEMBER

Custom Hook name	Must start with 'use' — useFetchMovies, useAuth, useSearch
Hook placement	Hooks always at TOP LEVEL — never inside if/loops/functions
onClick rule	onClick={fn} NOT onClick={fn()} — pass reference not call
API key safety	Always in .env file — add .env to .gitignore
key prop	Required in list rendering — key={movie.imdbID}
One job rule	Each file has ONE job — UI or logic, not both
BrowserRouter	Must wrap entire app for React Router to work
Link vs a tag	Always use <Link> not <a> inside React Router apps
await response.json()	Always needed after fetch — opens the envelope
Spread to copy	{...obj} creates real copy — obj2=obj1 is same reference